# Chapter 13: Operator Overloading

**ENG3091**

Khiem Nguyen

## Contents


**1 Introduction to operator overloading** **2**
- 1.1 Operators are functions . . . 2
- 1.2 Coming from Python: special methods . . . 2
- 1.3 In Python, special methods grant both wishes . . . 2
- 1.4 How the compiler sees an operator . . . 3
- 1.5 Without overloaded operators… . . . 3
- 1.6 …and with them . . . 4
- 1.7 Best practices (read at home) . . . 4

**2 Arithmetic operators as friend functions** **5**
- 2.1 Three ways to overload an operator . . . 5
- 2.2 Our example class: `Pennies` . . . 5
- 2.3 Overloading `+` and `-` . . . 6
- 2.4 The recipe . . . 6
- 2.5 A friend can be defined inside the class . . . 7
- 2.6 Operands of different types . . . 8
- 2.7 Implementing operators with other operators . . . 8

**3 Operators as normal functions** **9**
- 3.1 Friend or normal function? . . . 9
- 3.2 A normal-function operator . . . 9
- 3.3 Normal-function operators with a header . . . 10
- 3.4 Building it . . . 10

**4 Overloading the I/O operators** **11**
- 4.1 A `print()` function works, but… . . . 11
- 4.2 Overloading `operator<<` . . . 11
- 4.3 `operator<<` in action . . . 12
- 4.4 Why return `std::ostream&`? . . . 12
- 4.5 Overloading `operator>>` . . . 13

**5 Operators as member functions** **13**
- 5.1 Operators as member functions . . . 13
- 5.2 The friend version, again . . . 14
- 5.3 The member version . . . 15
- 5.4 Members for `Pennies + int`, friends for `<<` . . . 16
- 5.5 `+=` as a member, `+` as a normal function . . . 17
- 5.6 Friend, normal or member? (read at home) . . . 17

**6 Wrapping up** **18**
- 6.1 Summary . . . 18
- 6.2 Keep learning . . . 18

# 1 Introduction to operator overloading

## 1.1 Operators are functions

- Think of `1 + 2` as `add(1, 2)`: an operator is just a function with a special name and a convenient syntax.
- For our own `ComplexNumber` objects `a` and `b`, we'd like to write `a + b` instead of `add(a, b)` or `a.add(b)`.

In C++, operators really are implemented as functions, and we can **overload** them for our own types.

The words "in C++" matter: C and Java, for example, don't let you overload operators.

## 1.2 Coming from Python: special methods

**Listing 1** `complex_number_01.py`

```python
class ComplexNumber:
    def __init__(self, real=0, imag=0):
        self.real = real
        self.imag = imag


a = ComplexNumber(42, 24)
b = ComplexNumber(24, 42)
print(a)      # we'd like something meaningful here...
print(a + b)  # ...and we'd like Python to understand a + b
```

```
<__main__.ComplexNumber object at 0x7f3c2a1b2d50>
TypeError: unsupported operand type(s) for +: 'ComplexNumber' and 'ComplexNumber'
```

Python doesn't know how to print a `ComplexNumber`, or how to add two of them.

## 1.3 In Python, special methods grant both wishes

**Listing 2** `complex_number_02.py`

```python
class ComplexNumber:
    def __init__(self, real=0, imag=0):
        self.real = real
        self.imag = imag

    def __str__(self):  # used by print()
        return f"{self.real} + {self.imag}i"

    def __add__(self, other):  # used by +
        return ComplexNumber(self.real + other.real, self.imag + other.imag)


a, b = ComplexNumber(42, 24), ComplexNumber(24, 42)
print(f"a = {a}")
print(f"a + b = {a + b}")
```

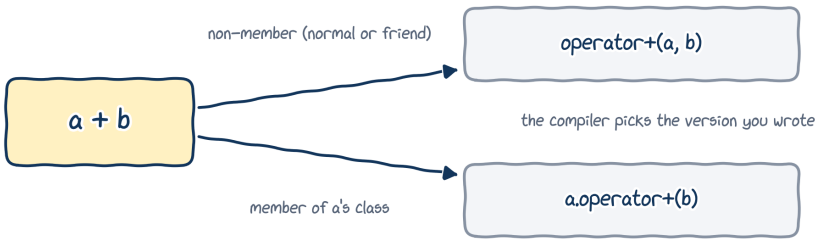
```
a = 42 + 24i
a + b = 66 + 66i
```

C++ does the same job with functions called `operator<<` and `operator+`.

## 1.4 How the compiler sees an operator

```cpp
int x{42};
int y{24};
std::cout << x + y << '\n'; // x + y means operator+(x, y)
```

With `double x` and `double y`, the same `x + y` calls the `double` version instead: plain **overload resolution** (Chapter 05).




## 1.5 Without overloaded operators...

**Listing 3** `operator_example_complex_number.cpp`

```cpp
#include <iostream>

class ComplexNumber {
  private:
    double real{}, imag{};

  public:
    ComplexNumber(double real, double imag) : real{real}, imag{imag} {}
};

int main() {
    ComplexNumber a{42, 24};
    ComplexNumber b{24, 42};
    std::cout << a + b << '\n'; // error: no + and no << for ComplexNumber
    return 0;
}
```

```
error: no match for 'operator+' (operand types are 'ComplexNumber' and 'ComplexNumber')
```

There's no built-in `+` for `ComplexNumber`: we have to tell the compiler what it means.

## 1.6 ...and with them

**Listing 4** `operator_example_complex_number_fixed.cpp`

```cpp
#include <iostream>

class ComplexNumber {
  private:
    double real{}, imag{};

  public:
    ComplexNumber(double real, double imag) : real{real}, imag{imag} {}

    ComplexNumber operator+(const ComplexNumber& other) const {
        return ComplexNumber{real + other.real, imag + other.imag};
    }

    friend std::ostream& operator<<(std::ostream& out, const ComplexNumber& z) {
        out << z.real << " + " << z.imag << 'i';
        return out;
    }
};

int main() {
    ComplexNumber a{42, 24};
    ComplexNumber b{24, 42};
    std::cout << a << '\n' << b << '\n' << a + b << '\n'; // all fine now
    return 0;
}
```

```
42 + 24i
24 + 42i
66 + 66i
```

This chapter explains every line of this program.

## 1.7 Best practices (read at home)

* Best practice

- An overloaded operator should work on at least one type **you** defined.
- Keep each operator's meaning as close to its usual meaning as possible: `+` should add.
- If the meaning wouldn't be clear and intuitive, write a named function instead.
- Operators that don't change their operands (like `+`) should return their result **by value**.
- Operators that change their left operand (like `+=` and `=`) should return that operand **by reference**.

# 2 Arithmetic operators as friend functions

## 2.1 Three ways to overload an operator

The arithmetic operators `+`, `-`, `*` and `/` take two operands, and all four are overloaded in exactly the same way. There are three ways to write them:

1. as a **friend** function;
2. as a **normal** (non-member, non-friend) function;
3. as a **member** function.

We start with friends, because they can read the private data of both operands.

## 2.2 Our example class: `Pennies`

**Listing 5** `class_pennies.cpp`

```
class Pennies {
  private:
    int pennies{};

  public:
    explicit Pennies(int pennies) : pennies{pennies} {}
    int get_pennies() const { return pennies; }
};
```

- **`explicit`** stops the compiler from turning an `int` into a `Pennies` behind our back: `Pennies p = 42;` and passing `42` where a `Pennies` is expected won't compile. A penny count is a whole number, and we want every conversion to be on purpose.
- You write `Pennies{42}` when you mean it.

## 2.3 Overloading + and -

**Listing 6** `overload_operators_plus_minus.cpp`

```cpp
#include <iostream>

class Pennies {
  private:
    int pennies{};

  public:
    explicit Pennies(int pennies) : pennies{pennies} {}
    int get_pennies() const { return pennies; }

    friend Pennies operator+(const Pennies& a, const Pennies& b);
    friend Pennies operator-(const Pennies& a, const Pennies& b);
};
Pennies operator+(const Pennies& a, const Pennies& b) {
    return Pennies{a.pennies + b.pennies};
}
Pennies operator-(const Pennies& a, const Pennies& b) {
    return Pennies{a.pennies - b.pennies};
}

int main() {
    Pennies mine{42};
    Pennies yours{24};
    std::cout << "Together: " << (mine + yours).get_pennies() << '\n';
    std::cout << "Difference: " << (mine - yours).get_pennies() << '\n';
    return 0;
}
```

```
Together: 66
Difference: 18
```

`operator+` and `operator-` are **not** members; they're friends, so they may use `a.pennies` and `b.pennies`.

## 2.4 The recipe

Overloading `+` is as simple as:

1. declaring a function named `operator+`;
2. giving it two parameters, one per operand (`const` references, since `+` doesn't change them);
3. picking a sensible return type (here, a new `Pennies` returned by value);
4. writing the body.

**Your turn:** how would you make all of these work?

```cpp
std::cout << (42 + mine).get_pennies();   std::cout << (mine + 42).get_pennies();
std::cout << (3 * mine).get_pennies();    std::cout << (mine * 3).get_pennies();
```

Start from `overload_operators_play_around.cpp`. My solution is in `overload_operators_play_around_solution.cpp` but try first!

## 2.5 A friend can be defined inside the class

**Listing 7** `friend_function_inside_class.cpp`

```cpp
#include <iostream>

class Pennies {
  private:
    int pennies{};

  public:
    explicit Pennies(int pennies) : pennies{pennies} {}
    int get_pennies() const { return pennies; }

    // Defined inside the class, but still a non-member function
    friend Pennies operator+(const Pennies& a, const Pennies& b) {
        return Pennies{a.pennies + b.pennies};
    }
};

int main() {
    Pennies mine{42};
    Pennies yours{24};
    Pennies sum{mine + yours};
    std::cout << "I have " << sum.get_pennies() << " pennies.\n";
    return 0;
}
```

```
I have 66 pennies.
```

As in Chapter 12: a friend defined inside the class is still a **non-member** function.

## 2.6 Operands of different types

**Listing 8** `vector3d_operands_of_different_types.cpp`

```cpp
#include <iostream>

class Vector3D {
  private:
    double x{}, y{}, z{};

  public:
    Vector3D(double x, double y, double z) : x{x}, y{y}, z{z} {}

    friend Vector3D operator+(double s, const Vector3D& v) { // s + v
        return Vector3D{s + v.x, s + v.y, s + v.z};
    }
    friend Vector3D operator+(const Vector3D& v, double s) { // v + s
        return s + v; // reuse the version above
    }

    void print() const {
        std::cout << "(" << x << ", " << y << ", " << z << ")\n";
    }
};

int main() {
    Vector3D w{1, 2, 3};
    (42 + w).print();
    (w + 42).print();
    return 0;
}
```

```
(43, 44, 45)
(43, 44, 45)
```

`42 + w` and `w + 42` are two **different** functions: the operands come in a different order. Overload both, just like any overloaded function.

## 2.7 Implementing operators with other operators

To make `w + 42` give the same answer as `42 + w`, the second operator simply **calls** the first:

```cpp
friend Vector3D operator+(const Vector3D& v, double s) {
    return s + v; // reuse operator+(double, const Vector3D&)
}
```

One real implementation, so there's only one place to fix if it's ever wrong.

# 3 Operators as normal functions

## 3.1 Friend or normal function?

We made `operator+` a friend because it needed the private member `pennies`. If a public getter gives it everything it needs, it doesn't have to be a friend:

```cpp
// As a friend: reads the private members directly
friend Pennies operator+(const Pennies& a, const Pennies& b) {
    return Pennies{a.pennies + b.pennies};
}

// As a normal function: uses the public getter
Pennies operator+(const Pennies& a, const Pennies& b) {
    return Pennies{a.get_pennies() + b.get_pennies()};
}
```

That's the whole idea.

## 3.2 A normal-function operator

**Listing 9** `overload_operators_with_normal_functions.cpp`

```cpp
#include <iostream>

class Pennies {
  private:
    int pennies{};

  public:
    explicit Pennies(int pennies) : pennies{pennies} {}
    int get_pennies() const { return pennies; } // public getter
};

// Neither a member nor a friend: the getter is all it needs
Pennies operator+(const Pennies& a, const Pennies& b) {
    return Pennies{a.get_pennies() + b.get_pennies()};
}

int main() {
    Pennies mine{42};
    Pennies yours{24};
    Pennies sum{mine + yours};
    std::cout << "I have " << sum.get_pennies() << " pennies.\n";
    return 0;
}
```

```
I have 66 pennies.
```

Friends and normal functions behave almost identically. One difference: a friend's declaration in the class doubles as its **prototype**; a normal function needs its own. That matters when we split files.

### 3.3 Normal-function operators with a header

**Listing 10** pennies/pennies.h

```
#ifndef PENNIES_H
#define PENNIES_H

class Pennies {
  private:
    int pennies{};

  public:
    explicit Pennies(int pennies) : pennies{pennies} {}
    int get_pennies() const { return pennies; }
};

// A normal function: other files only know it exists from this declaration
Pennies operator+(const Pennies& a, const Pennies& b);

#endif
```

**Listing 11** pennies/pennies.cpp

```
#include "pennies.h"

Pennies operator+(const Pennies& a, const Pennies& b) {
    return Pennies{a.get_pennies() + b.get_pennies()};
}
```

**Listing 12** pennies/main.cpp

```
#include "pennies.h"
#include <iostream>

int main() {
    Pennies mine{42};
    Pennies yours{24};
    Pennies sum{mine + yours}; // needs the declaration in pennies.h
    std::cout << "I have " << sum.get_pennies() << " pennies.\n";
    return 0;
}
```

### 3.4 Building it

```
$ cd pennies
$ g++ -std=c++20 -Wall main.cpp pennies.cpp -o pennies
$ ./pennies
I have 66 pennies.
```

- Without the declaration of `operator+` in `pennies.h`, `main.cpp` couldn't use `mine + yours`: it's compiled on its own (Chapter 02).
- If `main.cpp` lives in another folder, include the header by its relative path, as in `main_pennies.cpp`: `#include "pennies/pennies.h"`.

# 4 Overloading the I/O operators

## 4.1 A `print()` function works, but…

**Listing 13** `point_with_print.cpp`

```cpp
#include <iostream>

class Point {
  private:
    double x{}, y{}, z{};

  public:
    Point(double x = 0.0, double y = 0.0, double z = 0.0) : x{x}, y{y}, z{z} {}

    void print() const {
        std::cout << "Point(" << x << ", " << y << ", " << z << ')';
    }
};

int main() {
    const Point point{42.0, 24.0, 12.0};
    std::cout << "My point is ";
    point.print(); // breaks the statement up
    std::cout << " in Cartesian space.\n";
    return 0;
}
```

```
My point is Point(42, 24, 12) in Cartesian space.
```

`point.print()` breaks one output statement into three. We'd rather write `std::cout << point`, like Python's `__str__`.

## 4.2 Overloading `operator<<`

`<<` is a binary operator too, just like `+`. In `std::cout << point`:

- the left operand is `std::cout`, an object of type **`std::ostream`**;
- the right operand is our `Point`.

So our function looks like this:

```cpp
friend std::ostream& operator<<(std::ostream& out, const Point& p);
```

It can't be a member of `Point`: a member operator's left operand must be a `Point`. And it can't be a member of `std::ostream`, because we can't change the standard library.

## 4.3 `operator<<` in action

**Listing 14** `point_overload_output_operator.cpp`

```cpp
#include <iostream>

class Point {
  private:
    double x{}, y{}, z{};

  public:
    Point(double x = 0.0, double y = 0.0, double z = 0.0) : x{x}, y{y}, z{z} {}

    friend std::ostream& operator<<(std::ostream& out, const Point& p);
};

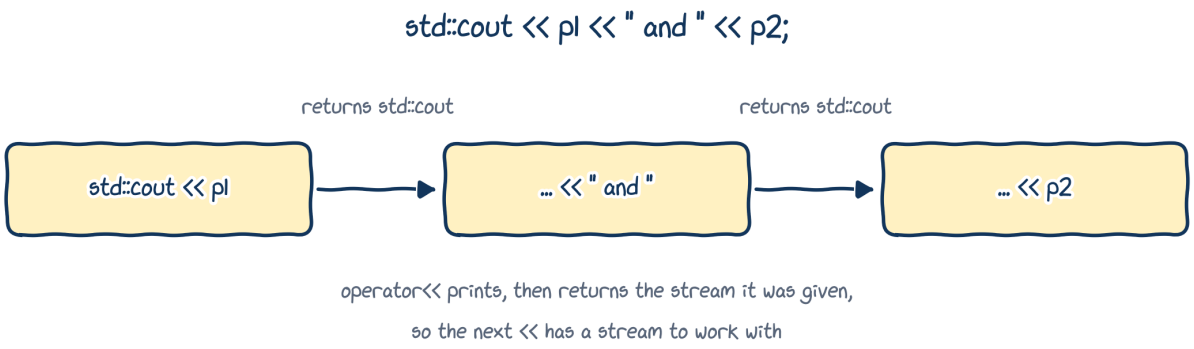
std::ostream& operator<<(std::ostream& out, const Point& p) {
    out << "Point(" << p.x << ", " << p.y << ", " << p.z << ')';
    return out; // return the stream, so that << can be chained
}

int main() {
    const Point p1{42.0, 24.0, 12.0};
    const Point p2{1.0, 2.0, 3.0};
    std::cout << "My points are " << p1 << " and " << p2 << '\n';
    return 0;
}
```

```
My points are Point(42, 24, 12) and Point(1, 2, 3)
```

Inside the function, `std::cout` is simply called `out`: a reference to whatever stream we're printing to.

## 4.4 Why return `std::ostream&`?




- **To allow chaining:** `operator<<` returns the stream it was given, so the next `<<` has a stream to work with. It's the same idea as returning `*this` in Chapter 12.
- **By reference:** `std::ostream` can't be copied, so returning it by value doesn't even compile.

### 4.5 Overloading `operator>>`

**Listing 15** `point_overload_input_operator.cpp`

```cpp
#include <iostream>

class Point {
  private:
    double x{}, y{}, z{};

  public:
    friend std::ostream& operator<<(std::ostream& out, const Point& p);
    friend std::istream& operator>>(std::istream& in, Point& p);
};

std::ostream& operator<<(std::ostream& out, const Point& p) {
    out << "Point(" << p.x << ", " << p.y << ", " << p.z << ')';
    return out;
}
std::istream& operator>>(std::istream& in, Point& p) { // p is NOT const
    in >> p.x >> p.y >> p.z;
    return in;
}

int main() {
    std::cout << "Enter a point: ";
    Point point{};
    std::cin >> point;
    std::cout << "You entered " << point << '\n';
    return 0;
}
```

```
$ ./point_overload_input_operator
Enter a point: 1 2 3
You entered Point(1, 2, 3)
```

The mirror image of `<<`: the left operand is `std::cin`, a **`std::istream`**, and the `Point` parameter is **not** `const`, because we read into it.

## 5 Operators as member functions

### 5.1 Operators as member functions

When an operator is overloaded as a **member** function:

- it must be a member of the class of the **left** operand;
- the left operand becomes the implicit object, `*this`;
- the other operand becomes the only parameter.

Converting a friend operator into a member is easy, and no new rules are needed: it's just an ordinary member function with a special name.

## 5.2 The friend version, again

**Listing 16** `overload_operator_plus_revisit.cpp`

```cpp
#include <iostream>

class Pennies {
  private:
    int pennies{};

  public:
    explicit Pennies(int pennies) : pennies{pennies} {}
    int get_pennies() const { return pennies; }

    friend Pennies operator+(const Pennies& a, const Pennies& b);
};

Pennies operator+(const Pennies& a, const Pennies& b) { // a friend
    return Pennies{a.pennies + b.pennies};
}

int main() {
    Pennies mine{42};
    Pennies yours{12};
    Pennies sum{mine + yours}; // operator+(mine, yours)
    std::cout << "I have " << sum.get_pennies() << " pennies.\n";
    return 0;
}
```

```
I have 54 pennies.
```

### 5.3 The member version

**Listing 17** `overload_operator_plus_member_function_01.cpp`

```cpp
#include <iostream>

class Pennies {
  private:
    int pennies{};

  public:
    explicit Pennies(int pennies) : pennies{pennies} {}
    int get_pennies() const { return pennies; }

    Pennies operator+(const Pennies& other) const; // a member now
};

// Defined outside the class: note the Pennies:: in front of the name
Pennies Pennies::operator+(const Pennies& other) const {
    return Pennies{pennies + other.pennies}; // pennies is this->pennies
}

int main() {
    Pennies mine{42};
    Pennies yours{12};
    Pennies sum{mine + yours}; // mine.operator+(yours)
    std::cout << "I have " << sum.get_pennies() << " pennies.\n";
    return 0;
}
```

```
I have 54 pennies.
```

`mine + yours` now means `mine.operator+(yours)`. A member function can be defined outside the class, with `Pennies::` in front of its name.

### 5.4 Members for `Pennies + int`, friends for `<<`

**Listing 18** `overload_operator_plus_member_function_02.cpp`

```cpp
#include <iostream>

class Pennies {
  private:
    int pennies{};

  public:
    explicit Pennies(int pennies) : pennies{pennies} {}

    Pennies operator+(int value) const { // a member: Pennies + int
        return Pennies{pennies + value};
    }

    // Must be a non-member: the left operand is a std::ostream
    friend std::ostream& operator<<(std::ostream& out, const Pennies& p) {
        return out << p.pennies << " pennies"; // << returns out
    }
};

int main() {
    Pennies mine{42};
    std::cout << "I have " << (mine + 12) << ".\n";
    return 0;
}
```

```
I have 54 pennies.
```

Watch out: `".\n"` (double quotes) is text. `'.\n'` (single quotes) would be a *multi-character literal*, which prints a strange number!

## 5.5 += as a member, + as a normal function

**Listing 19** `complex_plus_equal.cpp`

```cpp
#include <iostream>

class ComplexNumber {
  private:
    double real{}, imag{};

  public:
    ComplexNumber(double real, double imag) : real{real}, imag{imag} {}

    ComplexNumber& operator+=(const ComplexNumber& other) { // a member:
        real += other.real; // it changes *this...
        imag += other.imag;
        return *this; // ...and returns it
    }

    void print() const { std::cout << real << " + " << imag << "i\n"; }
};
ComplexNumber operator+(ComplexNumber a, const ComplexNumber& b) {
    return a += b; // a is a copy, so changing it is fine
}

int main() {
    ComplexNumber a{1, 2};
    ComplexNumber b{3, 4};
    (a + b).print();
    return 0;
}
```

```
4 + 6i
```

`+=` changes its left operand, so it's a member returning `*this` by reference. `+` is then a normal function that takes `a` **by value** (a copy) and applies `+=` to it.

## 5.6 Friend, normal or member? (read at home)

* Rules of thumb

- Assignment `=`, subscript `[]`, function call `()` and member selection `->` **must** be members: the language requires it.
- Unary operators (like `-x`): members.
- Binary operators that **don't change** their left operand (like `+`): normal functions (preferred) or friends.
- Binary operators that change their left operand, but whose left operand's class you can't change (like `<<` on `std::ostream`): normal functions or friends.
- Binary operators that change their left operand, in a class you can change (like `+=`): members.

# 6 Wrapping up

## 6.1 Summary

- An operator is a function with a special name: `a + b` calls `operator+(a, b)` or `a.operator+(b)`.
- Overload operators as **friend**, **normal** or **member** functions. A member operator belongs to the left operand's class, which becomes `*this`.
- `+` returns a new object by value; `+=` changes `*this` and returns it by reference. Implement one operator in terms of another.
- `operator<<` and `operator>>` take a stream (`std::ostream&` or `std::istream&`) as the left operand, and return it by reference to allow chaining.
- Use `explicit` constructors to prevent unwanted conversions.
- Keep each operator's usual meaning; if in doubt, write a named function.

## 6.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Overloading the arithmetic operators using friend functions
2. Overloading the I/O operators
3. Overloading operators using member functions
4. Overloading the comparison operators, including C++20's `<=>`

**Next up, Chapter 14:** inheritance.